RÉPUBLIQUE TUNISIENNE
Ministère de l'Enseignement Supérieur et de la Recherche Scientifique
UNIVERSITÉ PRIVÉE TEK-UP
École Supérieure des Technologies de l'Information et de la Communication



Spécialité SSIR — Sécurité des Systèmes d'Information et Réseaux

CONCEPTION ET RÉALISATION D'UN SERVEUR MCP INTELLIGENT DE THREAT INTELLIGENCE

Protection du secret, étiquetage et traçabilité de bout en bout

Rapport de projet — « Art de protection du secret » & « Cyber OPS »

Réalisé par	Mzabi Sirine · Khaled Ellif
Encadrant académique	Tarek Hdji
Validation 1	Architecture et exigences (P0 à P3)
Validation 2	Réalisation, durcissement et tests (P4–P5)

Année universitaire 2026-2027

Version 1.0 — 27 octobre 2026 · données synthétiques · laboratoire hors ligne

Logo officiel : tek-up.de — document académique, usage non commercial.

Résumé

Ce rapport documente la conception et la réalisation d'un serveur MCP de Threat Intelligence destiné à des agents LLM. Le serveur expose six outils en lecture seule, applique une politique ABAC fondée sur le rôle, la finalité, l'habilitation et les compartiments, étiquette chaque réponse, masque les champs protégés, neutralise les instructions hostiles et journalise chaque décision dans une chaîne d'audit vérifiable. Il fonctionne en stdio, sans réseau sortant, avec la bibliothèque standard de Python.

La campagne de validation comporte 31 scénarios, dont 16 abus : 31/31 réussis, 16/16 abus maîtrisés, latence p95 de 12,42 ms, révocation appliquée en 10 ms, audit complet (115/115 appels) et zéro faiblesse sur les huit contrôles comparés à une maquette naïve. Les limites sont explicites : données synthétiques, détection d'injection heuristique, journal local sans ancrage externe, posture déclarative et absence de transport réseau. Le rapport distingue systématiquement les éléments testés, configurés et proposés.

Mots-clés : Model Context Protocol, Threat Intelligence, protection du secret, ABAC, étiquetage, prompt injection, journal d'audit, moindre privilège, tests de sécurité.

> **Comment lire les statuts**
> « Testé » signifie qu'une commande a produit une preuve conservée. « Configuré » signifie que le mécanisme est en place mais que son intégration réelle reste à faire. « Proposé » signifie que la solution est décrite sans exécution. Aucun élément non testé n'est présenté comme fonctionnel.

Sommaire

Introduction générale 6
Ce que le lecteur peut vérifier 7
Partie I — Art de protection du secret 8
1. P0 — Cadrage du projet 8
1.1 Le problème réel, chiffré à notre échelle 8
1.2 Parties prenantes et attentes 8
1.3 Périmètre inclus et exclu 9
1.4 Cinq objectifs mesurables 9
1.5 Contraintes de réalisation 9
1.6 Thèse et fil rouge 9
1.7 Adéquation au sujet et apports propres 10
2. P1 — Modèle de menace et exigences 11
2.1 Actifs et niveaux de secret 11
2.2 Acteurs et surfaces d'attaque 11
2.3 Risques classés par probabilité et impact 12
2.4 Exigences fonctionnelles 12
2.5 Exigences non fonctionnelles 13
2.6 Lecture croisée risque → exigence → contrôle 13
3. P2 — Étude comparative des solutions 14
3.1 Critères, poids et justification 14
3.2 Quatre candidats 14
3.3 Notation pondérée et résultats 14
3.4 Décision, sensibilité et candidat le plus répandu 15
4. P3 — Architecture et points de contrôle 16
4.1 Trois zones de confiance et un flux non fiable 16
4.2 Circulation des étiquettes 16
4.3 Parcours d'un document classifié 17
4.4 Huit points de contrôle 18
4.5 Un seul point de journalisation, en échec fermé 18
4.6 Composants, rôles et exigences 18
4.7 Environnement de travail vérifié 19
4.8 Méthodologie par jalons et validations 19
4.9 Checklist de validation 1 19
Partie II — Cyber OPS 20
5. P4 — Réalisation, configuration et durcissement 20
5.1 Journal horodaté de la réalisation 20
5.2 Installation et configuration 20
5.3 Implémentation des contrôles 21
5.4 Durcissement du poste et du service 22
5.5 Données synthétiques et scénarios 22
5.6 État honnête de la réalisation 22
6. P5 — Tests de bout en bout et métriques 23
6.1 Protocole de test 23
6.2 Tests nominaux et de durcissement 23
6.3 Tests d'abus 24
6.4 Captures d'écran réelles du projet 24
6.5 Lecture des captures 27

6.6 Métriques et avant / après....28
6.7 Un échec réel, documenté et corrigé....28
6.8 Limites connues....29
7. Note de synthèse....29
7.1 Ce qui est testé....29
7.2 Ce qui n'est pas testé....29
7.3 Limites et travaux futurs....30
Conclusion générale....31
Bibliographie et références....32
Annexes....33
Annexe A — Catalogue des outils MCP....33
Annexe B — Codes d'audit et motifs....33
Annexe C — Procédure de reproduction....34
Annexe D — Unité systemd proposée....34
Annexe E — Glossaire....35

Table des figures

Figure 1 — Comparatif pondéré : contribution de chaque critère au score total.....15

Figure 2 — Architecture en zones de confiance, circulation des requêtes et huit points de contrôle.....16

Figure 3 — Parcours d'un bulletin classifié, de l'ingestion à la réponse et au refus.....17

Figure 4 — Environnement réel : Python 3.14.4, SQLite 3.46.1 et base de 12 bulletins, 30 indicateurs et 6 actifs.....25

Figure 5 — Découverte réelle du serveur : protocole 2025-06-18, six outils en lecture seule, neuf bulletins visibles.....26

Figure 6 — Fuite refusée deux fois : canari absent, code générique et motif interne distinct dans le journal.....26

Figure 7 — Même bulletin autorisé avec l'habilitation et le compartiment requis, puis instruction hostile neutralisée.....26

Figure 8 — Chaîne valide sur 116 entrées, puis rupture détectée à la séquence 31 sur la copie falsifiée.....27

Figure 9 — Comparaison réelle : canari et source visibles sans contrôle, absents ou masqués avec le serveur durci.....27

Figure 10 — Moindre privilège réel : uid 65534, base en 0440, clé en 0400 et écriture de la base refusée.....27

Figure 11 — Tableau de bord des métriques de la campagne.....28

Liste des tableaux

Tableau 1 — Chiffres clés de la campagne de validation....7
Tableau 2 — Apports propres et preuves associées....10
Tableau 3 — Actifs, classification et localisation....11
Tableau 4 — Risques, scores et contrôles principaux....12
Tableau 5 — Scores pondérés des quatre candidats....14
Tableau 6 — Niveaux, TLP et mentions de manipulation....17
Tableau 7 — Points de contrôle et exigences....18
Tableau 8 — Contrôles, implémentation et tests....21
Tableau 9 — Scénarios d'abus et résultats....24
Tableau 10 — Six outils MCP en lecture seule....33
Tableau 11 — Codes d'audit et motifs de décision....33
Tableau 12 — Glossaire des termes techniques....35

Introduction générale

Un analyste demande à un agent conversationnel : « résume le bulletin ADV-2026-004 ». L'agent appelle un outil MCP, reçoit un JSON, le place dans son contexte et rédige une synthèse. Si le serveur ne sait pas qui demande, pour quelle finalité et avec quelle habilitation, la réponse peut contenir un canari de détection et l'identité d'une source humaine. La fuite n'est pas une hypothèse : elle tient en un appel.

Ce rapport part de cette scène très concrète, et non d'une inquiétude générale sur la cybercriminalité. Le sujet est un serveur MCP (Model Context Protocol) de Threat Intelligence : un service qui expose des bulletins, des indicateurs de compromission et des scores de risque à des agents LLM. Le protocole MCP standardise le transport et la découverte d'outils ; il ne dit rien, par lui-même, de la confidentialité des données transportées. C'est précisément cet espace que le projet occupe : rendre la protection du secret démontrable à chaque appel, sans sortir du poste de travail.

Nous avons traité le sujet en deux temps liés par le même fil rouge. La première partie construit le cadrage, le modèle de menace, les exigences, le comparatif de solutions et l'architecture : c'est la matière de la validation 1. La seconde partie réalise le serveur dans un laboratoire WSL2, le durcit, puis exécute une campagne de 31 scénarios : c'est la matière de la validation 2. Les deux parties ne sont pas juxtaposées : chaque contrôle de la seconde répond à un risque nommé dans la première, et chaque test cite l'exigence qu'il valide.

La thèse que nous défendons tient en une phrase : **un serveur MCP de Threat Intelligence peut rester simple, local et hors ligne tout en rendant la protection du secret démontrable, à condition de traiter l'étiquette, l'habilitation et la trace comme des données de première classe à chaque appel.** Cette phrase guide les choix techniques : pas de base exposée au client, pas de réponse sans enveloppe étiquetée, pas de décision sans trace, pas de contenu partenaire exécuté comme une instruction.

Ce que le lecteur peut vérifier

Le rapport ne présente pas un prototype « prometteur » : il présente un artefact exécutable, ses preuves et ses limites. Toutes les valeurs chiffrées proviennent de la campagne du 27 octobre 2026, consignée dans `preuves/logs/tests_latest.json` et rejouable par `python3 tests/run_tests.py`. Les captures d'écran sont générées à partir de sorties réelles, et non dessinées pour l'illustration. Les statuts sont employés au sens strict : **proposé** signifie décrit mais non exécuté ; **configuré** signifie installé ou paramétré ; **testé** signifie exécuté avec une preuve reproductible.

Tableau 1 — Chiffres clés de la campagne de validation

Indicateur	Résultat	Ce que cela signifie
Scénarios exécutés	31 / 31	15 scénarios nominaux ou de durcissement et 16 scénarios d'abus.
Abus maîtrisés	16 / 16 (100 %)	Chaque tentative est refusée, neutralisée ou tracée sans fuite.
Latence p95	12,42 ms	Sur 60 requêtes locales, sans réseau, avec audit à chaque appel.
Révocation	10,0 ms	Le jeton révoqué est refusé à la requête suivante, sans redémarrage.
Audit	115 / 115 appels	Tous les appels attendus sont journalisés ; la chaîne est valide.
Avant / après	8 → 0 faiblesse	La maquette témoin échoue sur les huit contrôles comparés.
Injection	3 / 3 · 0 faux positif	Trois payloads détectés, aucun bulletin bénin signalé à tort.
Durcissement	uid 65534	Le service tourne sous « nobody », base 0440, clé 0400.

Source : preuves/logs/tests_latest.json, exécution du 27 octobre 2026 à 13:16 UTC.

Le rapport est organisé selon les cinq phases demandées. La partie I couvre P0 à P3 et se termine par une checklist de validation 1. La partie II couvre P4 et P5, avec le journal horodaté, les tests, les métriques, l'échec intermédiaire corrigé et les limites. Une note de synthèse, une conclusion, une bibliographie et cinq annexes complètent le dossier. Nous avons préféré des tableaux de preuve compacts et des captures lisibles, et systématiquement indiqué la source de chaque chiffre.

> **Avertissement de périmètre**
>
> Toutes les données sont synthétiques : adresses RFC 5737, domaines `.example`, familles et sources fictives. Aucune attaque n'a été menée hors du laboratoire. Les canaris sont des leurres créés pour le projet. Les fonctions de production non testées sont explicitement marquées « proposé » ou « à vérifier ».

Partie I — Art de protection du secret

Du problème réel à l'architecture : cadrage, modèle de menace, exigences, comparatif et conception.

Cette partie construit la décision avant de construire le code. Nous commençons par formuler le problème en termes mesurables, puis nous identifions les actifs, les acteurs et les risques. Les exigences sont ensuite dérivées des risques, et non inventées à côté. Le comparatif justifie le choix d'un serveur dédié plutôt que d'une plateforme commerciale ou d'une passerelle générique. L'architecture, enfin, montre où circulent les étiquettes, où sont placés les points de contrôle et pourquoi il n'existe qu'un seul point de journalisation.

1. P0 — Cadrage du projet

1.1 Le problème réel, chiffré à notre échelle

Le problème n'est pas l'absence de renseignement : il est dans le passage du renseignement vers un agent qui ne comprend ni les niveaux de classification, ni le besoin d'en connaître. Un bulletin Secret peut contenir un canari — une chaîne unique qui permet de détecter une fuite — et l'identité d'une source. Une réponse trop riche, un message d'erreur trop précis ou une recherche non filtrée suffisent à les exposer. Le coût d'une fuite n'est pas seulement technique : il inclut la perte de la source, la rupture de confiance avec le partenaire et, à l'échelle d'une organisation, un coût moyen de **4,44 M$** pour une violation de données en 2025 selon IBM [1].

Nous avons voulu une mesure propre à notre laboratoire, plutôt qu'un chiffre emprunté. La maquette témoin `tests/baseline_naive.py` expose les mêmes données sans contrôle d'accès, sans assainissement et sans journal. Sur les huit contrôles comparés, elle échoue **huit fois sur huit** : elle livre le canari d'un bulletin Secret sans jeton, laisse l'identité de source en clair, transmet une instruction hostile telle quelle, accepte une requête surdimensionnée, ne sépare pas les rôles, ne journalise rien, ne peut pas révoquer et ne limite pas le débit. Le serveur durci échoue zéro fois sur ces mêmes huit contrôles. Ce n'est pas une projection : c'est un différentiel mesuré, reproductible et présenté au chapitre 6.

Le problème est donc double. D'un côté, un agent LLM est un **délégué confus** : il exécute ce qu'on lui demande, y compris une instruction cachée dans un contenu importé. De l'autre, un serveur de renseignement est naturellement tenté d'être généreux : plus il renvoie de contexte, plus l'agent semble utile. La protection du secret exige l'inverse : renvoyer le minimum nécessaire, au bon niveau, avec une trace.

1.2 Parties prenantes et attentes

Sept parties prenantes structurent le projet : l'analyste SOC attend rapidité et réponses étiquetées ; le responsable CTI, un audit et une révocation immédiate ; le RSSI et le délégué à la protection des données, la preuve de minimisation ; l'intégrateur d'agents, un contrat d'outil stable ; le partenaire, une source jamais

exposée ; l'auditeur ou le jury, des preuves reproductibles ; l'exploitant, une installation sans dépendance ni exposition réseau. Ces attentes convergent vers trois exigences : six outils en lecture seule, des décisions tracées et des refus génériques.

1.3 Périmètre inclus et exclu

Inclus : transport MCP stdio (JSON-RPC 2.0, protocole 2025-06-18), six outils en lecture seule et ressources filtrées, jetons HMAC-SHA256 avec rôles, finalités, habilitation 0–3 et compartiments, étiquetage et TLP, masquage de champ, canaris, refus génériques, neutralisation d'injections, journal append-only chaîné, exécution sous « nobody » avec base en lecture seule et absence de client réseau, jeu synthétique de 12 bulletins, 30 IOC et 6 actifs.

Exclus ou laissés à l'intégration : transport HTTP, OAuth 2.1 et mTLS, écriture et connecteurs de flux commerciaux, annuaire d'entreprise et fédération d'identité, chiffrement applicatif et HSM, classification sémantique par modèle, ancrage externe du journal et SIEM, EDR réel et chiffrement de disque, données et campagnes réelles. Ces éléments sont repris comme travaux futurs dans la note de synthèse.

1.4 Cinq objectifs mesurables

Cinq objectifs mesurables ont guidé la conception : **O1**, aucune réponse classifiée sans droits (16/16 abus maîtrisés, canari absent des refus) ; **O2**, chaque décision traçable et chaîne vérifiable (115/115 appels, 116 entrées) ; **O3**, contenu partenaire hostile neutralisé et signalé (3/3, 0 faux positif sur 11 bénins) ; **O4**, service utilisable (p95 12,42 ms sur 60 requêtes, cible ≤ 50 ms) ; **O5**, accès révocable immédiatement et service non privilégié (10,0 ms, uid 65534, base 0440). Le détail figure au chapitre 6.

1.5 Contraintes de réalisation

Quatre contraintes ont réellement pesé sur l'architecture : aucune donnée réelle ni flux commercial (jeu synthétique étiqueté), aucune attaque hors du laboratoire (scénarios rejoués localement), bibliothèque standard uniquement sous Python 3.14.4 (stdio, sqlite3, hmac, hashlib, json ; venv réservé aux documents) et aucun privilège root (exécution sous « nobody », base 0440, clé 0400). Deux contraintes de contexte — validation le jour de la soutenance, poste WSL2 sans systemd garanti — ont conduit à une architecture légère en trois zones, lancée par script, avec une unité systemd seulement proposée. La traçabilité exigée par le sujet explique enfin la matrice risque → exigence → contrôle et la colonne « statut » de chaque test.

1.6 Thèse et fil rouge

> *« Un serveur MCP de Threat Intelligence peut rester simple, local et hors ligne tout en rendant la protection du secret démontrable, à condition de traiter l'étiquette, l'habilitation et la trace comme des données de première classe à chaque appel. »*
>
> — Thèse du projet

Cette thèse a une conséquence de conception : les contrôles ne sont pas des options ajoutées après coup. Ils sont dans le chemin obligatoire de la réponse. Une réponse ne peut pas être produite sans étiquette, sans décision d'habilitation et sans entrée d'audit. C'est ce que nous appelons un **chemin de réponse contrôlé**, par opposition à une bibliothèque de fonctions que chaque outil pourrait contourner.

> **Ce qui reste ouvert à ce stade**
>
> Le raccordement à un agent EDR réel, à un annuaire d'entreprise et à un ancrage externe du journal relève de l'intégration, pas de la maquette. Ces points sont assumés et repris dans les limites.

1.7 Adéquation au sujet et apports propres

Le sujet demandé est le « développement d'un serveur MCP intelligent de Threat Intelligence ». Les trois termes sont couverts sans détour. **Serveur MCP** : le binaire implémente le protocole 2025-06-18 sur stdio, répond à `initialize`, `tools/list`, `tools/call`, `resources/list`, `resources/read` et `ping`, et expose six outils en lecture seule. **Threat Intelligence** : il sert des bulletins, des indicateurs, des acteurs, des actifs et des scores CVSS, EPSS et KEV, avec corrélation par famille de malware. **Intelligent** : le score composite est expliqué, pondéré par la confiance et la fraîcheur, et le serveur détecte les instructions hostiles glissées dans un bulletin partenaire. Ce n'est donc pas un CRUD renommé : c'est un service d'analyse avec des décisions de sécurité opposables.

L'apport du projet n'est pas un nouvel algorithme. Il est dans la combinaison, à notre connaissance peu documentée, entre un serveur MCP et une chaîne de protection du secret vérifiable à chaque appel. Les serveurs MCP que nous avons étudiés exposent des outils avec une clé d'API et rendent le JSON tel quel ; notre serveur place l'étiquette, l'habilitation, le masquage et la trace dans le chemin obligatoire de la réponse. C'est cette contrainte de passage qui produit la différence, et c'est elle que les tests d'abus cherchent à contourner.

Tableau 2 — Apports propres et preuves associées

Apport	Pratique étudiée	Notre serveur	Preuve
Accès au secret	Clé d'API, tout ou rien	Treillis Public → Secret, compartiments, besoin d'en connaître, posture de poste	A06, A07, EF-05 à EF-08
Étiquette dans la réponse	JSON brut, classification hors bande	Étiquette, masquage de champs et canaris dans chaque réponse	A10, EF-12
Contenu partenaire	Transmis tel quel au modèle	Instruction hostile rendue inerte, signalée et journalisée	A12, A15, 3/3
Traçabilité	Journal applicatif non chaîné	Chaîne d'empreintes en échec fermé, 115 appels sur 115 tracés	A14, ENF-03
Reproductibilité	Démonstration manuelle	31 scénarios, données synthétiques, laboratoire hors ligne	31/31, preuves/

Source : étude comparative du chapitre 3 et campagne du chapitre 6.

Nous qualifions cette innovation d'**ingénierie et d'intégration**, pas de rupture scientifique. Elle est défendable parce qu'elle est mesurée : 16 abus sur 16 maîtrisés, zéro fuite de canari, audit complet et révocation immédiate. Elle est aussi limitée : détection d'injection heuristique, pas de transport HTTP, pas d'ancrage externe du journal. Le chapitre 7 nomme ces limites et leur condition de levée.

2. P1 — Modèle de menace et exigences

Le modèle de menace est spécifique au sujet : un serveur de renseignement qui parle à un agent LLM. Il ne s'agit pas de lister toutes les menaces informatiques, mais de nommer ce qui peut faire perdre le secret dans ce montage précis. Nous partons des actifs, puis des acteurs et des surfaces, avant de classer les risques par probabilité et impact.

2.1 Actifs et niveaux de secret

Nous utilisons quatre niveaux : **Public (0)**, **Interne (1)**, **Confidentiel (2)** et **Secret (3)**, assortis d'une conversion TLP pour l'échange. Le niveau ne suffit pas : un acteur peut être Secret et ne pas avoir le compartiment CRYPTO. Le besoin d'en connaître est donc modélisé comme un second axe, indépendant du niveau. Le tableau ci-dessous donne les actifs que nous protégeons et la raison de leur classement.

Tableau 3 — Actifs, classification et localisation

ID	Actif	Niveau / TLP	Pourquoi ce niveau	Où il vit
A1	Bulletins Secret (attribution, opération CANARI)	Secret · TLP:RED	Une divulgation peut brûler une source et un leurre.	SQLite, réponse filtrée
A2	Identité des sources humaines	Secret · TLP:RED	Le champ `source_identity` relie une information à une personne.	Champ de bulletin, masqué
A3	Bulletins Confidentiels (vulnérabilités non corrigées)	Confidentiel · TLP:AMBER	Une lecture anticipée donne un avantage à l'attaquant.	SQLite
A4	Indicateurs et familles de malware	Interne · TLP:GREEN	Utile au SOC, sensible hors du cercle.	SQLite
A5	Actifs, criticité et exposition	Confidentiel · TLP:AMBER	La carte des actifs critiques est une cible.	SQLite
A6	Clé de signature des jetons	Secret opérationnel	Sa compromission permet de forger des capacités.	Fichier 0400, hors base
A7	Journal d'audit	Confidentiel · intégrité critique	Il prouve les décisions ; le modifier efface la responsabilité.	JSONL append-only
A8	Métadonnées de santé	Public à Interne	Utile à l'exploitation, ne doit pas révéler les volumes Secret.	Réponse `ti_health`

Source : jeu de données synthétique du laboratoire ; les canaris sont des chaînes de contrôle.

2.2 Acteurs et surfaces d'attaque

Nous distinguons les acteurs légitimes, dont les droits varient, des acteurs hostiles ou ambigus. L'analyste SOC (habilitation 1–2) cherche et enrichit ; le lead CTI (niveau 3 et compartiments) lit les bulletins Secret de son périmètre ; le lecteur ou auditeur consulte un périmètre restreint. L'agent LLM occupe une place à part : il est légitime, mais **manipulable**. Il peut recevoir une instruction cachée dans un bulletin partenaire et la transformer en appel d'outil — c'est la délégation confuse. Le partenaire fournit des bulletins et peut empoisonner la base ; l'attaquant vole un jeton, forge une signature ou sature le service ; un poste non conforme (BYOD) compromet le point d'accès au secret. C'est pourquoi le serveur ne fait jamais confiance au contenu qu'il renvoie : il l'enveloppe comme une donnée non exécutable.

Les surfaces d'attaque sont peu nombreuses, ce qui est un avantage : le serveur n'écoute aucun port, ne sort pas sur le réseau et n'expose que six outils. Les surfaces retenues sont le canal stdio, les paramètres d'outil, les URI de ressources, le contenu importé, le fichier de jeton, la base, la clé, le journal et l'attestation de posture. Chacune est rattachée à un point de contrôle dans l'architecture (chapitre 4).

2.3 Risques classés par probabilité et impact

Nous avons retenu dix risques, notés de 1 à 5 sur deux axes. Le score brut P × I sert au premier classement ; quand deux risques sont à égalité, l'impact tranche avant la probabilité, car une perte de source est moins réversible qu'une tentative répétée sans conséquence. Le classement est donné ci-dessous.

Tableau 4 — Risques, scores et contrôles principaux

ID	Risque	P	I	Score	Contrôle principal	Exigences
R01	Fuite d'un bulletin Secret par une réponse trop riche.	4	5	20	ABAC, étiquette de sortie, masquage, canari.	EF-03, EF-04
R02	Injection de consignes dans un flux partenaire.	4	5	20	Neutralisation, enveloppe non exécutable, fanions.	EF-05, ENF-12
R03	Jeton forgé, rejoué ou élevé en privilège.	3	5	15	HMAC, posture, rôle, révocation, audit.	EF-02, EF-09
R04	Accès hors besoin d'en connaître.	4	4	16	Compartiments, ressources filtrées, refus générique.	EF-03, EF-11
R05	Fuite par message d'erreur ou énumération d'identifiants.	4	3	12	Refus identiques, validation stricte, aucun oracle.	EF-03, EF-12
R06	Altération ou suppression des traces.	3	5	15	Chaîne SHA-256, append-only, vérification.	EF-08, ENF-08
R07	Poste client compromis ou non conforme.	4	4	16	Attestation de posture, refus explicite, moindre privilège.	EF-02, ENF-05
R08	Saturation du service par un client.	4	3	12	Limiteur par sujet, fenêtre glissante.	EF-13, ENF-09
R09	Empoisonnement de la base de renseignement.	3	5	15	Assainissement à l'ingestion, validation, score explicable.	EF-05, EF-07
R10	Fichiers, base ou clé laissés accessibles.	3	4	12	Base 0440, clé 0400, service non privilégié.	ENF-06, ENF-07

Source : modèle de menace du projet ; scores assumés et justifiés, non mesurés.

Deux risques dominent : la fuite par réponse trop riche (R01) et l'injection par flux partenaire (R02). Ils ont le même score, mais ils n'appellent pas les mêmes contrôles. R01 se traite par la décision d'accès et l'enveloppe de réponse ; R02 se traite à l'ingestion et à la présentation. Le fait de les séparer nous a évité de croire qu'un filtre d'injection suffirait à protéger un bulletin Secret, ou qu'un contrôle d'accès suffirait à neutraliser une instruction cachée.

2.4 Exigences fonctionnelles

Treize exigences fonctionnelles décrivent ce que le serveur doit **faire** ; chacune cite les risques qu'elle réduit et le test qui la vérifie. Le catalogue expose six outils en lecture seule avec schémas stricts (EF-01, testé N01). Chaque appel est authentifié par jeton signé et posture (EF-02, A01–A05). La décision combine rôle, finalité, habilitation et compartiments, avec refus sans oracle (EF-03, A06–A09, A15) ; l'étiquette de sortie masque les champs protégés (EF-04, N03, N04, N10). L'assainissement neutralise les instructions hostiles (EF-05, A10) et la recherche résiste à l'injection SQL (EF-06, N02, N05, N06, A13). L'enrichissement, la corrélation et le score restent explicables (EF-07, N07–N09).

Chaque décision est journalisée (EF-08, N03, A01–A05, A14) et la révocation s'applique à la requête suivante (EF-09, A04, N13). L'état de santé ne divulgue aucun contenu (EF-10, N01, N11), les ressources sont filtrées (EF-11, N10, A15), les entrées invalides sont refusées (EF-12, N12, A09, A11, A16) et le débit est limité (EF-13, A12). Toutes sont **testées** au 27 octobre 2026.

2.5 Exigences non fonctionnelles

Douze exigences non fonctionnelles, toutes mesurées au 27 octobre 2026 : latence p95 ≤ 50 ms (12,42 ms sur 60 requêtes) ; 100 % des abus refusés ou neutralisés sans fuite (16/16) ; 100 % des appels attendus journalisés (115/115) ; révocation appliquée à la requête suivante en ≤ 5 s (10,0 ms) ; aucun client réseau sortant et aucun secret dans l'état ; exécution sous uid 65534 ; base 0440 et clé 0400 ; détection de toute altération du journal ; robustesse aux entrées invalides ; portabilité sans dépendance hors bibliothèque standard ; survie de la révocation et de la chaîne au redémarrage ; injection 3/3 avec 0 faux positif sur 11 bénins. Chaque seuil a été fixé avant l'exécution, pas après.

2.6 Lecture croisée risque → exigence → contrôle

La traçabilité est portée par le tableau des risques et par les exigences détaillées ci-dessus : le tableau associe à chaque R0x un contrôle principal et des exigences, et chaque exigence cite les risques réduits et le test qui la vérifie. La lecture croisée fait apparaître un choix important : le contrôle d'accès couvre à lui seul quatre risques, mais ni l'injection ni l'intégrité des traces. Nous avons donc refusé de considérer l'ABAC comme la réponse unique au problème. La protection du secret est un empilement de barrières indépendantes : si l'une cède, les autres ne s'effondrent pas avec elle.

3. P2 — Étude comparative des solutions

Avant de coder, nous avons comparé quatre façons de rendre un service de Threat Intelligence accessible à un agent LLM. La comparaison n'est pas un concours de popularité : elle part de critères justifiés, puis note chaque candidat, puis assume une décision. Les notes sont des jugements argumentés sur une échelle de 1 à 5 ; elles ne prétendent pas être des mesures de laboratoire. Nous expliquons donc aussi ce qui se passerait si les poids changeaient.

3.1 Critères, poids et justification

Les critères sont ordonnés avant de voir les résultats, pour éviter de choisir les poids en fonction du candidat préféré. Le critère de sécurité pèse le plus lourd parce que le sujet est la protection du secret ; le protocole et l'adéquation fonctionnelle viennent ensuite, car un serveur sûr mais inutilisable ne rend pas service. La maîtrise et le coût sont des contraintes de projet, pas des qualités intrinsèques.

Cinq critères et leurs poids sont fixés avant la notation. **Sécurité du secret, 30 %** : une fuite annule l'utilité du service ; indicateurs observés, refus, masquage, audit et moindre privilège. **Conformité MCP, 20 %** : l'agent doit découvrir et appeler les outils sans adaptation fragile ; initialize, tools/list, stdio, ressources. **Adéquation TI, 20 %** : le service doit aider un SOC ; couverture des six outils et des étiquettes. **Maîtrise et réversibilité, 15 %** : le projet doit rester compréhensible, réparable et portable ; dépendances, lisibilité, absence de verrou. **Coût et délai, 15 %** : aucun achat n'est possible ; licences, infrastructure, temps d'intégration. Les poids ne bougeront plus ; la section 3.4 vérifie seulement ce qui se passerait s'ils changeaient.

3.2 Quatre candidats

Quatre candidats couvrent un spectre réaliste. **C1** est un script Python local : immédiat, sans dépendance, mais sans contrôle, sans trace et sans protocole MCP. **C2** est une plateforme TI commerciale (MISP, OpenCTI ou équivalent) avec un connecteur MCP : riche fonctionnellement, mais les données sortent du poste, les licences et l'intégration sont lourdes. **C3** est une passerelle MCP générique : conforme au protocole et réutilisable, mais elle ne comprend ni les étiquettes, ni les compartiments, ni le besoin d'en connaître. **C4** est un serveur MCP dédié : stdio, six outils, ABAC, étiquettes, audit et données locales. Nous avons inclus les deux extrêmes (C1 et C2), car ce sont les choix les plus fréquents — le script dans un projet étudiant, la plateforme dans une entreprise.

3.3 Notation pondérée et résultats

Chaque candidat reçoit une note de 1 à 5 par critère. Le score total est la somme des notes pondérées. Le tableau donne les contributions, pas seulement le total, afin que le lecteur puisse contester un point précis plutôt que le résultat global.

Tableau 5 — Scores pondérés des quatre candidats

Candidat	Sécurité 30 %	MCP 20 %	TI 20 %	Maîtrise 15 %	Coût 15 %	Total / 5
C1 — Script local naïf	0,15	0,10	0,40	0,75	0,75	2,15
C2 — Plateforme commerciale	1,05	0,20	0,90	0,30	0,23	2,68
C3 — Passerelle MCP	0,90	0,90	0,10	0,53	0,60	3,03
C4 — Serveur dédié (retenu)	1,35	0,90	0,80	0,68	0,45	4,18

Source : notation assumée sur une échelle de 1 à 5 ; les notes brutes sont visibles dans le graphique.

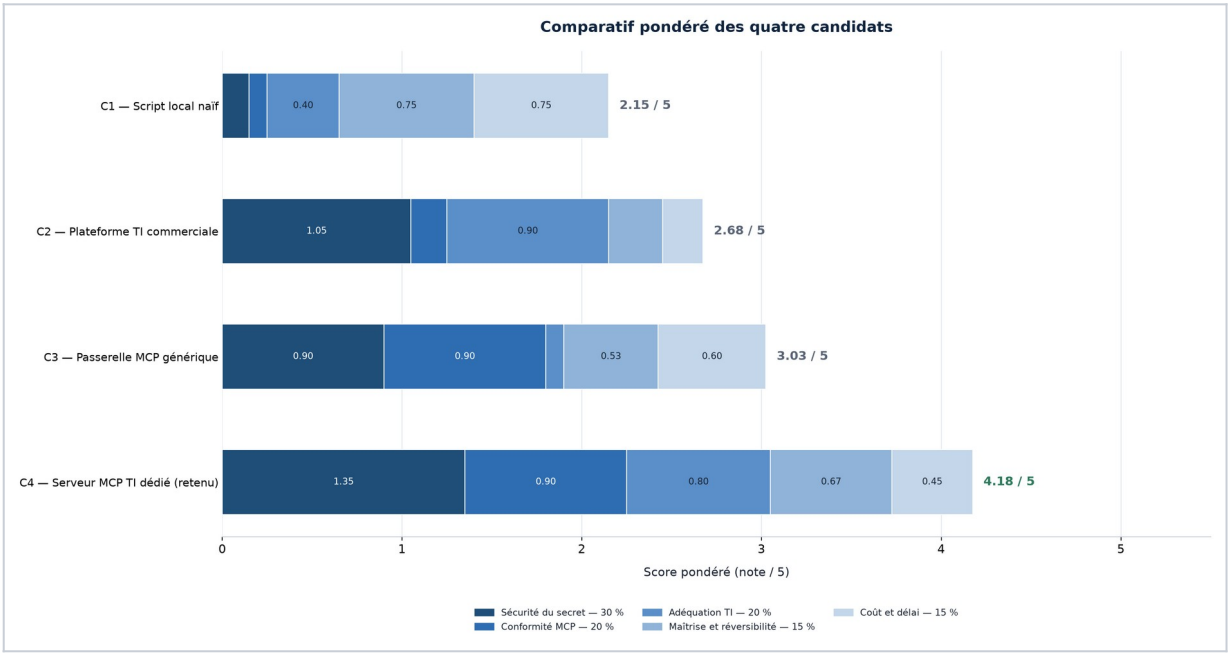


Figure 1 — Comparatif pondéré : contribution de chaque critère au score total

Source : mêmes notes que la notation ci-dessus ; C4 est retenu avec 4,18 / 5.

3.4 Décision, sensibilité et candidat le plus répandu

Le serveur dédié (C4) arrive en tête avec **4,18 / 5**, devant la passerelle MCP (3,03), la plateforme commerciale (2,68) et le script naïf (2,15). Le résultat ne repose pas sur un seul critère : C4 gagne parce qu'il est le seul à cumuler une sécurité démontrable, une conformité MCP native et une autonomie locale. La passerelle est meilleure que C4 sur le coût, mais elle ne comprend pas les étiquettes ; la plateforme est meilleure sur la richesse fonctionnelle, mais elle expose les données et coûte cher ; le script est le plus rapide à écrire, mais il ne protège rien.

Nous avons testé la sensibilité du classement à deux variantes : sécurité portée à 40 % (MCP 15 %, TI 15 %, maîtrise 15 %, coût 15 %) et coût porté à 40 % (sécurité 20 %, MCP 15 %, TI 15 %, maîtrise 10 %). Dans les deux cas, C4 reste premier (4,20 / 5 et 3,83 / 5). Le classement est donc robuste aux deux critiques les plus prévisibles : « vous privilégiez la sécurité » et « vous sous-estimez le coût ».

Le candidat **le plus répandu** mérite une explication séparée. Dans un projet étudiant, le plus fréquent est C1 : un script local, immédiat, sans dépendance. Nous l'excluons parce qu'il ne peut pas démontrer la protection du secret : il n'a ni jeton, ni étiquette, ni journal, et il livre le canari en clair. Dans l'industrie, le plus fréquent est C2 : une plateforme TI éprouvée, avec un connecteur. Nous l'excluons pour trois raisons : les données sortent du poste, l'intégration dépasse le temps disponible et la démonstration hors ligne devient impossible. Ce n'est pas un jugement sur ces solutions en production ; c'est un jugement sur leur adéquation au périmètre et aux contraintes de ce projet.

> **Décision assumée**
>
> Nous retenons C4 : un serveur MCP dédié, local, en bibliothèque standard, avec six outils en lecture seule, une politique ABAC, des étiquettes, un journal chaîné et une campagne de tests. Les notes sont des jugements argumentés, pas des mesures ; la sensibilité du classement est explicitée ci-dessus.

4. P3 — Architecture et points de contrôle

L'architecture répond à une question simple : où peut-on perdre le secret, et où le empêche-t-on ? Nous avons organisé le système en trois zones de confiance et un flux partenaire non fiable, puis placé huit points de contrôle sur les frontières. Le client ne parle jamais directement aux données ; le serveur ne sort jamais du poste ; tout ce qui entre ou sort passe par un point nommé.

4.1 Trois zones de confiance et un flux non fiable

La zone 0 est le poste de l'analyste : l'agent LLM, le client MCP, le jeton et l'attestation de posture. La zone 1 est le processus serveur, exécuté sans privilège : authentification, décision, outils, assainissement, enveloppe, limiteur et journal. La zone 2 contient les actifs : base SQLite en lecture seule, clé de signature, journal. La zone 3 est le flux partenaire : un contenu non fiable, ingéré hors ligne, validé et neutralisé avant stockage. Cette séparation est volontairement simple : trois zones tiennent sur une diapositive et se vérifient sur le poste.

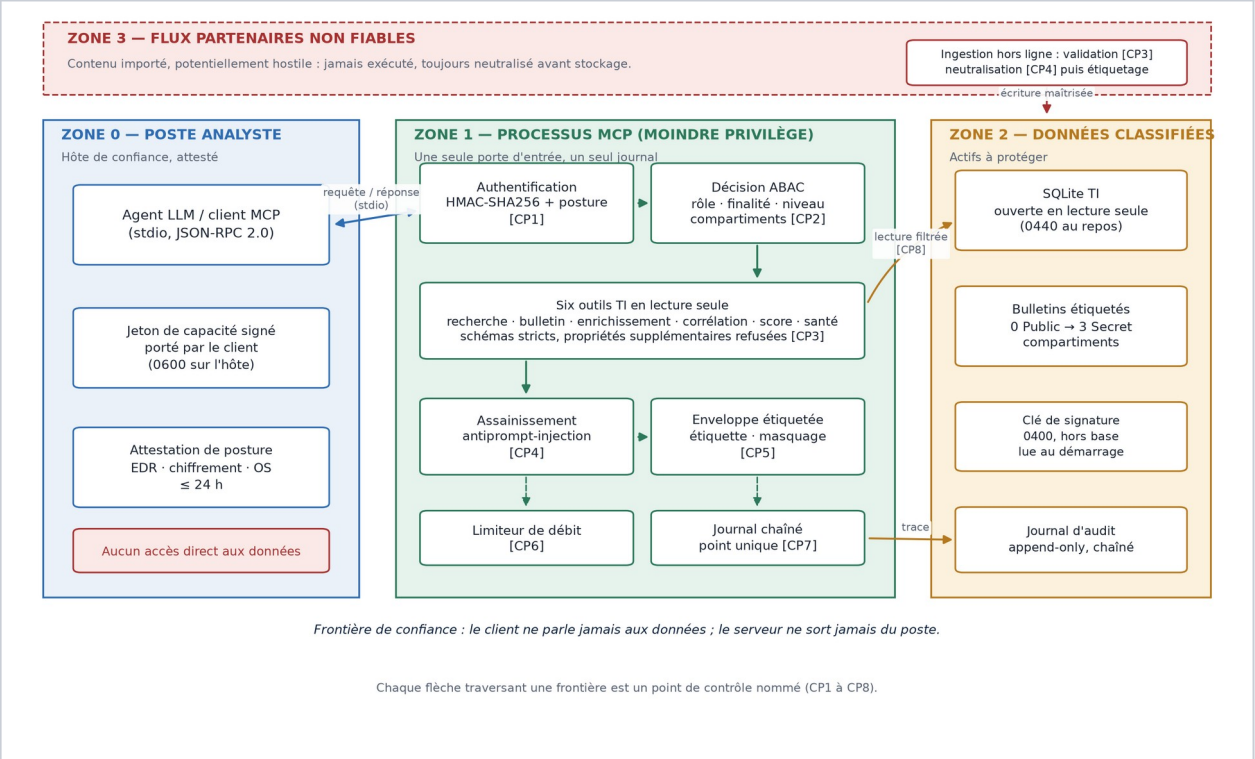


Figure 2 — Architecture en zones de confiance, circulation des requêtes et huit points de contrôle
Source : conception du projet ; CP1 à CP8 sont détaillés au §4.4.

Deux flèches méritent d'être lues ensemble. La flèche « requête / réponse » traverse la frontière de la zone 0 vers la zone 1 : c'est le seul canal d'entrée. La flèche « lecture filtrée » va de la zone 1 vers la zone 2 : elle est en lecture seule et passe par CP8. Il n'existe aucune flèche directe de la zone 0 vers la zone 2. C'est cette absence qui rend la compromission du client insuffisante, à elle seule, pour lire la base.

4.2 Circulation des étiquettes

Une étiquette n'est pas un commentaire : c'est une donnée qui voyage avec la réponse et qui détermine ce que le client a le droit d'en faire. Nous utilisons un triplet **(niveau, TLP, compartiments)**. Le niveau va de 0 à 3 ; le TLP reprend CLEAR, GREEN, AMBER et RED ; les compartiments sont des ensembles de besoins d'en connaître (CTI, CRYPTO, ATTRIB, INC-2026-014). Une réponse prend le niveau maximal des documents qu'elle contient, et elle ajoute une mention de manipulation.

Tableau 6 — Niveaux, TLP et mentions de manipulation

Niveau	Nom	TLP associé	Mention de manipulation	Exemple
0	Public	TLP:CLEAR	Diffusion libre après validation.	Bulletin de sensibilisation.
1	Interne	TLP:GREEN	Usage interne au laboratoire.	Indicateurs de famille.
2	Confidentiel	TLP:AMBER	Diffusion limitée au besoin d'en connaître.	Vulnérabilité non corrigée.
3	Secret	TLP:RED	Lecture strictement nominative ; ne pas citer.	Opération CANARI, source humaine.

Source : politique de classification du projet ; la conversion TLP est une convention de laboratoire.

La règle de masquage est volontairement asymétrique. L'identité de source (`source_identity`) n'est visible qu'au niveau 3 ; en dessous, elle est remplacée par une valeur absente et le champ est signalé dans `champs_masques`. Le canari, lui, n'est présent que dans un bulletin Secret et n'apparaît jamais dans un refus. Cette asymétrie permet un test positif (le Secret autorisé voit le canari) et un test négatif (le Confidentiel ne le voit pas), sans révéler l'existence de la ressource.

4.3 Parcours d'un document classifié

Le parcours complet comporte huit étapes. Les quatre premières concernent l'ingestion et le stockage ; les quatre suivantes concernent la demande et la réponse. Le refus est une branche de l'étape 6 : il ne dit jamais si la ressource existe, mais il laisse une trace aussi précise qu'une autorisation.

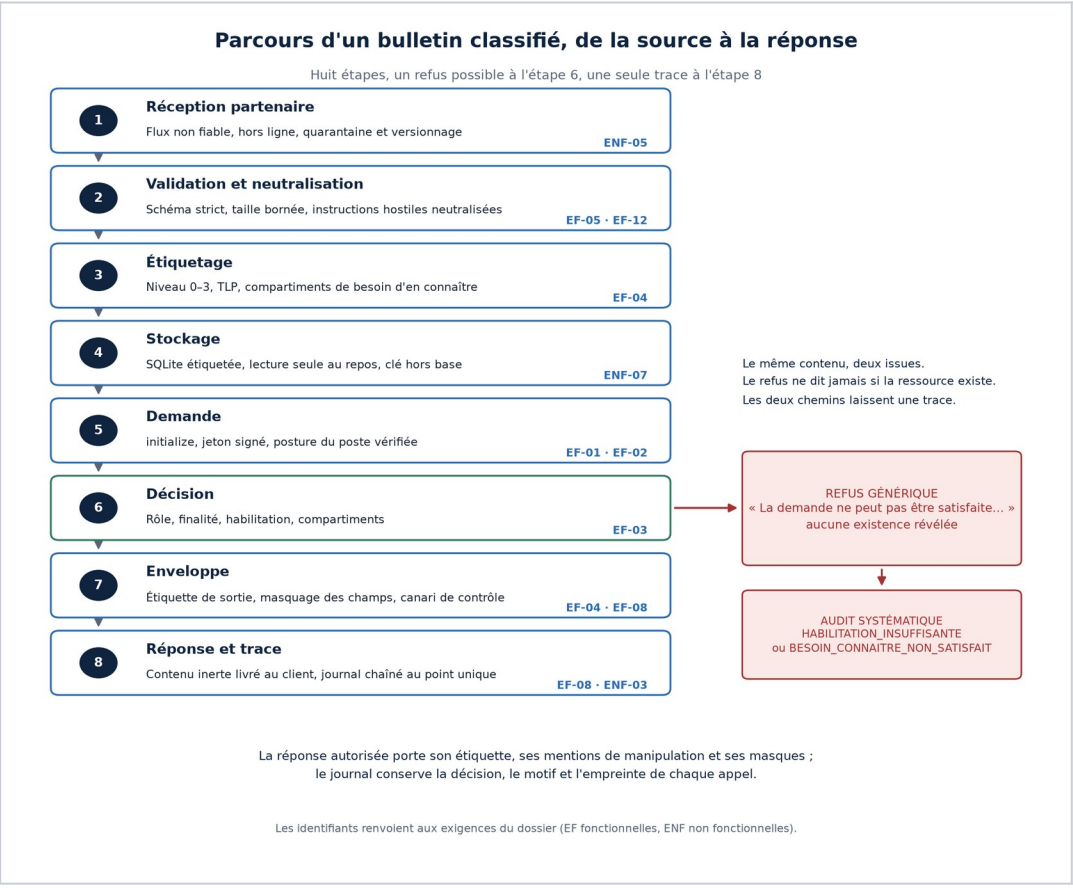


Figure 3 — Parcours d'un bulletin classifié, de l'ingestion à la réponse et au refus

Source : conception du projet ; chaque étape cite les exigences qu'elle met en œuvre.

Ce parcours met en évidence un point souvent négligé : le refus n'est pas une absence de traitement. Il faut décider, journaliser et répondre de façon générique. Si le refus est plus lent, plus bavard ou plus ambigu qu'une réponse autorisée, il devient un oracle. C'est pourquoi les tests A06, A07 et A08 comparent les

messages et les temps de réponse, et pourquoi l'audit conserve un motif interne distinct alors que le client ne reçoit qu'un code générique.

4.4 Huit points de contrôle

Les points de contrôle sont nommés CP1 à CP8. Ils sont placés sur des frontières, pas dans des fonctions arbitraires : chacun correspond à un moment où une donnée change de zone ou de nature. Cette nomenclature nous a servi à la fois pour coder et pour expliquer le système à l'oral.

Tableau 7 — Points de contrôle et exigences

Point	Nom	Ce qu'il vérifie	Exigences
CP1	Authentification et posture	Signature HMAC, expiration, révocation, EDR, chiffrement, OS, attestation ≤ 24 h.	EF-02, ENF-04
CP2	Décision ABAC	Rôle, finalité, niveau, compartiments, existence de la ressource.	EF-03, EF-11
CP3	Validation des entrées	Schéma strict, types, taille, profondeur, propriétés interdites.	EF-12, ENF-09
CP4	Assainissement	Détection et neutralisation des instructions, balises actives, exfiltration.	EF-05, ENF-12
CP5	Enveloppe et masquage	Étiquette de sortie, TLP, mentions, champs masqués, canari.	EF-04, ENF-02
CP6	Limite de débit	Fenêtre glissante par sujet, refus explicite sans effet de bord.	EF-13
CP7	Journal unique	Séquence, empreinte, motif, détail, fanions, latence ; échec fermé.	EF-08, ENF-03, ENF-08
CP8	Accès données	Lecture seule, filtrage, moindre privilège, modes de fichiers.	ENF-06, ENF-07

Source : architecture ; chaque point est implémenté dans un module identifié au chapitre 5.

4.5 Un seul point de journalisation, en échec fermé

Le journal est volontairement unique. Chaque appel, qu'il soit autorisé ou refusé, produit exactement une entrée dans `preuves/lab/audit.jsonl`. L'entrée contient la séquence, l'empreinte précédente, l'empreinte courante, l'horodatage, le sujet, l'outil, la décision, le motif interne, le détail, les fanions d'injection, les champs masqués et la latence. L'empreinte est calculée sur la concaténation de l'empreinte précédente et du contenu canonique de l'entrée : toute modification rompt la chaîne.

Le point important n'est pas la chaîne elle-même, mais sa position dans le chemin de réponse. Si l'écriture d'audit échoue sur une réponse destinée à être autorisée, le serveur refuse la réponse. C'est le **fail-closed** : mieux vaut ne pas répondre que répondre sans trace. Nous avons testé ce comportement par construction et par la vérification de la chaîne ; la limite est qu'un attaquant disposant des droits d'écriture sur le fichier peut réécrire l'ensemble. Sans ancrage externe, la chaîne détecte les altérations locales, pas une réécriture complète et cohérente.

4.6 Composants, rôles et exigences

Le code est organisé en modules courts, chacun avec une responsabilité lisible. `config.py` porte les constantes, niveaux, TLP, rôles, finalités et seuils ; `auth.py`, l'émission et la vérification des jetons HMAC ; `labels.py`, l'analyse des étiquettes et la couverture des compartiments ; `policy.py`, l'authentification, la posture, l'ABAC, le masquage et les refus génériques. `sanitize.py` détecte et neutralise les injections ; `store.py` gère SQLite, l'ingestion et la lecture seule ; `intel.py` l'enrichissement, la décroissance de confiance, la corrélation et le score ; `tools.py` expose les six outils et leurs schémas stricts ; `server.py` porte la boucle MCP, le limiteur, l'audit, les ressources et le fail-closed ; `audit.py` tient le journal append-

only et sa chaîne ; ti_cli.py fournit l'initialisation, les jetons, la révocation, l'audit et les statistiques. Tous ces modules sont **testés** au 27 octobre 2026.

Cette séparation n'est pas de l'architecture décorative : elle permet de tester un contrôle sans démarrer tout le serveur, et de montrer au jury un fichier précis pour chaque exigence.

4.7 Environnement de travail vérifié

Les prérequis ne sont pas déclarés : ils sont vérifiés par une commande et recopiés ci-dessous avec leur sortie observée le 27 octobre 2026. Si une version change, la colonne « observé » doit être revalidée.

Les prérequis ont été vérifiés par commande le 27 octobre 2026. python3 --version renvoie Python 3.14.4 ; SQLite est en 3.46.1 ; nobody a l'uid 65534 et setpriv est présent ; TI_DB=... python3 src/ti_cli.py stats compte 12 bulletins, 30 IOC et 6 actifs ; verify-audit renvoie ok=true sur 116 entrées. Ces sorties proviennent du conteneur WSL2 (noyau 6.18.54.1-microsoft-standard-WSL2, LibreOffice 26.2.6.3, poppler 26.01.0) ; si une version change, la valeur observée doit être revalidée.

Une nuance importante : les modes affichés pour la base et la clé ci-dessus sont ceux de l'état de travail (640/600, propriétaire root). Le test N15 recopie les fichiers, les passe en 0440/0400 et lance le serveur sous nobody ; c'est cette exécution qui prouve le moindre privilège. Nous ne confondons pas l'état de travail et l'état durci.

4.8 Méthodologie par jalons et validations

Le projet a suivi sept jalons (J0 à J6) alignés sur les deux validations. **J0** cadre le problème, les parties prenantes, le périmètre, les objectifs, les contraintes et la thèse. **J1** construit le modèle de menace, les risques R01–R10 et les exigences EF/ENF. **J2** compare quatre candidats et justifie C4. **J3** fixe l'architecture : zones, étiquettes, parcours, CP1–CP8, journal et composants, et passe la validation 1. **J4** réalise le serveur, les six outils, l'ABAC, l'audit, la CLI et les données synthétiques. **J5** durcit et exécute la campagne de 31 scénarios avec captures, métriques et comparatif avant/après. **J6** produit le rapport, le deck, le conducteur et la note. Chaque jalon a un livrable vérifiable et ne commence pas si le précédent n'est pas relu.

4.9 Checklist de validation 1

La validation 1 ne demande pas de démonstration d'exécution : elle demande une architecture défendable. Les points attendus se lisent directement dans le plan : problème chiffré et périmètre (§1.1–1.5), actifs et risques notés P × I (§2.1–2.3), exigences EF/ENF et lecture croisée (§2.4–2.6), comparatif pondéré et sensibilité (§3.1–3.4), architecture en zones, étiquettes, parcours, CP1–CP8 et journal (§4.1–4.6), environnement vérifié et jalons (§4.7–4.8). Chaque point renvoie à une section et à une preuve du chapitre 6, ce qui permet au jury de vérifier sans parcourir tout le rapport.

> **Validation 1 — statut**
> L'architecture est complète, cohérente avec les risques et prête à être défendue. Les points ouverts sont explicitement listés : agent EDR réel non raccordé, ancrage externe du journal non réalisé, transport HTTP/OAuth non implémenté. Aucun de ces points n'est nécessaire pour démontrer la protection du secret dans le laboratoire.

Partie II — Cyber OPS

Réalisation, durcissement, tests de bout en bout et preuves : de l'architecture au serveur qui tourne.

Cette partie ne raconte pas une installation idéale. Elle raconte ce qui a été exécuté, dans l'ordre, avec les commandes, les sorties et les échecs. Le serveur tourne sous WSL2, sans réseau sortant, avec des données synthétiques. Les tests sont rejouables et les captures proviennent des sorties. Quand quelque chose n'a pas été testé, nous l'écrivons.

5. P4 — Réalisation, configuration et durcissement

La réalisation a suivi l'architecture, module par module, en gardant le serveur exécutable à chaque étape. Nous avons d'abord fait fonctionner le canal MCP et les six outils, puis ajouté les contrôles dans l'ordre du chemin de réponse : authentification, décision, validation, assainissement, enveloppe, audit. Le durcissement est venu ensuite, sans changer les contrats d'outil.

5.1 Journal horodaté de la réalisation

Le journal ci-dessous reprend les étapes réelles, avec l'heure locale du laboratoire et la preuve associée. Il ne s'agit pas d'un planning théorique : les heures correspondent aux fichiers produits et aux exécutions conservées. L'échec de 13:14 est volontairement présent ; il est analysé au chapitre 6.

Les étapes réelles, avec l'heure locale et la preuve associée : **12:40** arborescence et configuration (config.py, labels.py) ; **12:52** données synthétiques (12 bulletins, 30 IOC, 6 actifs, 2 canaris, 1 injection) ; **13:02** boucle stdio, initialize, tools/list et six outils ; **13:08** premiers tests, avec un échec réel de l'audit sur la deuxième écriture ; **13:14** correction du décodage UTF-8 sur un flux texte dans audit._last ; **13:16** campagne complète (31/31, 16/16 abus, p95 12,42 ms) ; **13:21** démonstration isolée (fuite bloquée, Secret autorisé, injection neutralisée) ; **14:21** huit captures et six figures ; **14:40** rapport, deck et conducteur. Les heures correspondent aux fichiers de preuve conservés ; l'échec de 13:14 est volontairement présent et analysé au chapitre 6.

5.2 Installation et configuration

Le serveur n'a aucune dépendance d'exécution. Cette propriété est vérifiée par l'analyse statique des imports et par l'absence de client HTTP. Le venv ne sert qu'aux outils de document (python-docx, python-pptx, Pillow, matplotlib, pypdf) ; il n'est pas utilisé par le serveur. La procédure ci-dessous part d'un poste WSL2 vierge et produit le laboratoire de tests.

```
cd /root/tekup-mcp-ti
python3 --version                # Python 3.14.4
export TI_HOME=$PWD
```

```
export TI_DATA=$PWD/data
export TI_DB=$PWD/preuves/lab/ti.db
export TI_KEY_PATH=$PWD/preuves/lab/token.key
export TI_AUDIT=$PWD/preuves/lab/audit.jsonl
python3 src/ti_cli.py init-db --force
python3 src/ti_cli.py stats
python3 src/ti_cli.py verify-audit --path preuves/lab/audit.jsonl
```

La sortie observée de `init-db` est un objet JSON avec `advisories: 12`, `iocs: 30`, `assets: 6`, et la répartition 4 Public / 3 Interne / 3 Confidentiel / 2 Secret. La commande `stats` confirme ces volumes. La vérification d'audit renvoie `ok: true`, `entries: 116`, `broken_at: null`. Ces trois sorties suffisent à savoir si le laboratoire est dans l'état attendu avant une démonstration.

L'émission d'un jeton se fait par la CLI, avec un sujet, un rôle, une habilitation, des compartiments, une finalité et une posture. Le jeton est signé en HMAC-SHA256 et stocké dans un fichier 0600. La commande `inspect-token` permet de vérifier les claims sans révéler la signature ; `revoke` ajoute le `jti` à la liste de révocation, relue à chaque requête.

```
python3 src/ti_cli.py issue-token \
  --sub agent-soc-01 --role analyst --clearance 2 \
  --compartments CTI,INC-2026-014 --purpose triage \
  --out preuves/tokens/smoke.tok
python3 src/ti_cli.py inspect-token --token-file preuves/tokens/smoke.tok
python3 src/ti_cli.py revoke --jti <jti>
```

Le serveur se lance par `scripts/run_server.sh`, qui fixe `PYTHONPATH` et exécute `python3 -m ti_server`. Il parle JSON-RPC 2.0 sur l'entrée et la sortie standard, une ligne par message. Aucun port n'est ouvert. Cette propriété est essentielle pour le laboratoire : elle réduit la surface d'attaque et rend la démonstration possible sans réseau.

5.3 Implémentation des contrôles

Chaque contrôle de l'architecture correspond à un module et à un test. Le tableau ci-dessous est le pont entre la conception et le code ; il permet de répondre à la question « où est-ce implémenté ? » sans parcourir tout le dépôt.

Tableau 8 — Contrôles, implémentation et tests

Contrôle	Implémentation	Point clé	Test
Authentification	auth.verify_token, policy.authenticate	HMAC, expiration, révocation, posture.	A01–A05
ABAC et compartiments	policy.authenticate, policy.can_read	Rôle, finalité, niveau, ensembles.	A06–A09, A15
Validation stricte	tools.validate_arguments	Schéma, types, taille, propriétés interdites.	N12, A11, A16
Assainissement	sanitize.scan_injection, neutralize	Regex, zéro-largeur, lignes neutralisées.	A10
Enveloppe	tools._envelope	Étiquette max, TLP, mentions, masquage, canari.	N03, N04, N10
Audit	audit.AuditLog, server._audit_event	Chaîne, séquence, fail-closed.	A14, N13
Révocation	ti_cli revoke, policy.authenticate	Liste relue à chaque requête.	A04, N13
Débit	server.RateLimiter	Fenêtre glissante par sujet.	A12
Moindre privilège	Lancement setpriv, modes de fichiers	uid 65534, base 0440, clé 0400.	N15

Source : code du serveur ; les noms de fonctions sont exacts.

5.4 Durcissement du poste et du service

Le durcissement n'est pas un supplément esthétique : il réduit l'impact d'une compromission du processus. Nous avons appliqué quatre mesures vérifiables : exécution sous nobody, base en lecture seule, clé accessible au seul propriétaire, absence de client réseau. La commande de lancement utilisée par le test N15 est reproduite ci-dessous.

```
/usr/bin/setpriv --reuid=65534 --regid=65534 --clear-groups \
  /usr/bin/python3 -m ti_server
```

Le test N15 vérifie que le service démarre sous uid 65534, que la base est en 0440 et la clé en 0400, que le journal reste accessible en écriture et qu'une tentative d'écriture dans la base est refusée. Le résultat est uid=65534, base=0o440, clé=0o400, audit=True, écriture_refusée=True. C'est une preuve de moindre privilège, pas une preuve d'isolation par espace de noms : ce dernier test n'a pas été exécuté, et nous le disons.

L'analyse statique vérifie l'absence de client réseau et de dépendance d'exécution : douze fichiers, aucun import interdit (requests, urllib, http.client, socket côté serveur). Cette analyse ne remplace pas un test d'isolation réseau, mais elle est reproductible et suffisante pour démontrer l'absence de code d'exfiltration dans le périmètre.

Une unité systemd est proposée en annexe, avec User=nobody, NoNewPrivileges=yes, ProtectSystem=strict, PrivateNetwork=yes et ReadOnlyPaths. Elle n'est pas installée dans le laboratoire, faute de systemd garanti dans WSL2 ; elle est donc marquée **proposé**, pas **testé**. Cette distinction est importante : un jury peut demander à voir l'unité, mais nous ne prétendons pas l'avoir exécutée.

> **Ce qui n'est pas durci dans la maquette**
>
> Le chiffrement de disque est attesté par le jeton, mais aucun agent réel ne le vérifie. La clé est protégée par les permissions du système de fichiers, pas par un HSM. Le journal est local et non ancré à l'extérieur. Le transport est stdio ; un transport HTTP exigerait TLS, mTLS et une limitation de débit en amont. Ces points sont des travaux d'intégration, pas des contrôles cachés.

5.5 Données synthétiques et scénarios

Le jeu de données contient 12 bulletins, 30 indicateurs et 6 actifs. Quatre bulletins sont Public, trois Interne, trois Confidentiel et deux Secret. Les 30 indicateurs se répartissent entre les familles SableFin, NuitBleue, RougeMer et Canari ; le champ de source d'identité est masqué sous le niveau 3. Deux canaris permettent de tester la fuite : CANARY-TEKUP-7f3a9c-2026 dans ADV-2026-004 et CANARY-TEKUP-b41d8e-2026 dans ADV-2026-008. Un bulletin, ADV-2026-012, contient des instructions hostiles et une balise active. Les six actifs ont une criticité et une exposition variables, agrégées dans un score composite. Les adresses utilisent la plage de documentation RFC 5737 et les domaines .example : aucune donnée réelle n'est présente.

5.6 État honnête de la réalisation

Tous les contrôles décrits dans cette section ne sont pas au même degré de maturité. Nous distinguons trois statuts, sans mélange. **Testé** (preuve conservée) : six outils MCP (N01, N02, N07–N09), ABAC et compartiments (A06–A09, A15), étiquettes et masquage (N03, N04, N10), injection neutralisée (A10, 3/3, 0 faux positif), audit chaîné (A14, 116 entrées), révocation (A04, N13, 10 ms) et moindre privilège (N15, uid 65534, 0440/0400). **Configuré** (mécanisme en place, intégration réelle à faire) : posture de l'agent EDR (claim synthétique, aucun agent réel raccordé) et chiffrement de disque (attesté par le jeton, non vérifié par le serveur). **Proposé** (décrit, non exécuté) : unité systemd fournie en annexe mais non installée, OAuth/mTLS de production hors périmètre stdio, ancrage externe du journal nécessaire contre une réécriture complète. Aucun statut « testé » n'est attribué sans preuve.

6. P5 — Tests de bout en bout et métriques

La campagne de tests est le cœur de la validation 2. Elle comporte 31 scénarios : 15 nominaux ou de durcissement, 16 d'abus. Chaque scénario définit ses préconditions, ses étapes, son résultat attendu, son résultat obtenu et les exigences qu'il valide. Aucun test n'est supprimé lorsqu'il échoue : l'échec est documenté, corrigé, puis rejoué. Le lecteur peut relancer la campagne avec une seule commande.

```
cd /root/tekup-mcp-ti
python3 tests/run_tests.py
# Sortie attendue (extrait) :
# Tests: 31/31 — échecs: 0
# Détection abus: 16/16 (100.0 %)
# Latence p95: 12.42 ms (n=60)
# Audit: {'valide': True, 'entrees': 116, 'appels_attendus': 115, 'completude_pct': 100.0}
```

6.1 Protocole de test

Le protocole est volontairement simple, pour être compris par un jury non spécialiste et rejoué sans interprétation. Le harnais recrée un laboratoire propre, émet neuf jetons (analyste, Secret, faible, mauvais compartiment, poste non conforme, expiré, révoqué, limite de débit, auditeur, redémarrage), démarre un serveur par scénario, appelle les outils en stdio et lit la dernière entrée d'audit. Il compare ensuite le résultat à l'attendu et écrit une ligne de preuve. Les tests sont indépendants : un échec n'empêche pas les suivants, et le serveur est relancé pour chaque profil.

Trois précautions rendent la campagne crédible. D'abord, les jetons sont signés avec une clé de laboratoire et les scénarios d'abus utilisent de vrais refus, pas des simulations. Ensuite, les assertions portent sur le contenu (présence du canari, code d'erreur, motif d'audit), pas seulement sur le code HTTP ou le texte. Enfin, la campagne vérifie la chaîne d'audit après les tests, pour s'assurer que les contrôles n'ont pas seulement l'air corrects : ils ont laissé une trace cohérente.

6.2 Tests nominaux et de durcissement

Les quinze tests nominaux vérifient que le service fonctionne et que les contrôles ne cassent pas l'usage légitime. Ils couvrent l'initialisation, la recherche, le masquage, l'accès Secret, les compartiments, l'enrichissement, la corrélation, le score, les ressources, la santé, la validation, le redémarrage, l'analyse statique et l'exécution non privilégiée.

Les quinze tests nominaux réussissent tous. **N01** initialise le protocole 2025-06-18 et découvre six outils en lecture seule ; **N02** renvoie neuf bulletins à un analyste, sans Secret ; **N03** masque `source_identity` sur un bulletin Secret ; **N04** montre le canari uniquement dans le cercle Secret ; **N05** sépare deux périmètres (INC voit 003 et 011, CRYPTO voit 004 et 008) ; **N06** ne rend que quatre bulletins Public à une faible habilitation ; **N07** applique la décroissance de confiance (0,94 → 0,758) ; **N08** corrèle SableFin sur onze indicateurs sans inclure de canari ; **N09** produit un score de 9,24 en priorité P1 en excluant le bulletin Secret ; **N10** liste neuf ressources, masque 003 et refuse 004 ; **N11** expose l'état du service sans contenu sensible ; **N12** refuse deux entrées invalides et reste vivant ; **N13** conserve une révocation après redémarrage avec une chaîne valide ; **N14** ne trouve aucun client réseau ni dépendance hors bibliothèque standard ; **N15** tourne sous `nobody`, avec base en 0440 et clé en 0400. La source de ces résultats est `preuves/logs/tests_latest.json`.

N13 et N15 sont les deux tests qui font passer le durcissement du discours à la preuve : persistance de la révocation, et moindre privilège vérifié par une écriture refusée.

6.3 Tests d'abus

Les seize tests d'abus couvrent les risques R01 à R10. Ils ne cherchent pas à « casser » le serveur : ils vérifient qu'une tentative produit le comportement attendu, sans fuite et avec une trace. Chaque ligne du tableau indique le code observé et l'exigence validée.

Tableau 9 — Scénarios d'abus et résultats

ID	Scénario d'abus	Attendu	Obtenu	Statut
A01	Appel sans jeton	Réponse isError, code AUTH_REFUSEE, trace avec sujet inconnu.	code=AUTH_REFUSEE, audit=TOKEN_INVALIDE	testé
A02	Jeton expiré	Refus AUTH_REFUSEE ; audit TOKEN_EXPIRE.	code=AUTH_REFUSEE, audit=TOKEN_EXPIRE	testé
A03	Signature de jeton altérée	Refus AUTH_REFUSEE ; audit SIGNATURE_INVALIDE.	code=AUTH_REFUSEE, audit=SIGNATURE_INVALIDE	testé
A04	Jeton révoqué à chaud	Premier appel autorisé, second refusé ; révocation visible à la requête suivante.	commande=121.3 ms, application=10.0 ms, code=AUTH_REFUSEE, audit=TOKEN_REVOQUE	testé
A05	Poste non conforme	Refus POSTURE_NON_CONFORME ; audit explicite.	code=POSTURE_NON_CONFORME, audit=POSTURE_NON_CONFORME	testé
A06	Tentative de fuite par habilitation insuffisante	Refus générique, canari absent, audit HABILITATION_INSUFFISANTE.	code=RESSOURCE_INDISPONIBLE, canari_present=False, audit=HABILITATION_INSUFFISANTE	testé
A07	Tentative de fuite par défaut de cloisonnement	Refus générique, canari absent, audit BESOIN_CONNAITRE_NON_SATISFAIT.	code=RESSOURCE_INDISPONIBLE, canari_present=False, audit=BESOIN_CONNAITRE_NON_SATISFAIT	testé
A08	Énumération directe d'identifiants (IDOR)	Même code et même message pour les deux : aucune oracle d'existence.	codes=RESSOURCE_INDISPONIBLE/RESSOURCE_INDISPONIBLE, messages_identiques=True	testé
A09	Dépassement de rôle et de finalité	Deux refus ACCES_REFUSE, aucune donnée d'actif retournée.	lecteur=ACCES_REFUSE (OUTIL_NON_AUTORISE), auditeur=ACCES_REFUSE (OUTIL_NON_AUTORISE)	testé
A10	Injection d'instructions dans un flux partenaire	Contenu neutralisé, fanions d'injection, aucune instruction brute, canari absent.	fanions=['appel_outil', 'balise_active', 'exfiltration', 'ignorer_instructions'], instruction_brute=False, canari=False	testé
A11	Outil inconnu et paramètre interdit	Refus ACCES_REFUSE et ENTREE_INVALIDE ; serveur toujours disponible.	outil_inconnu=ACCES_REFUSE, paramètre=ENTREE_INVALIDE, vivant=True	testé
A12	Limite de débit	Six autorisés, deux refusés LIMITE_DEBIT.	autorisés=6, refusés=2, code=LIMITE_DEBIT	testé
A13	Contournement de recherche	Aucune erreur serveur, aucun résultat Secret, aucune fuite.	sql=0, identifiant_secret=0, canari_absent=True	testé
A14	Falsification du journal d'audit	La copie est détectée, l'original reste valide.	original=True (44), copie_ok=False, rupture_seq=31	testé
A15	Lecture directe d'une ressource Secret	Erreur -32002, aucune ressource retournée, audit BESOIN/HABILITATION.	erreur=-32002, audit=HABILITATION_INSUFFISANTE	testé
A16	Message JSON malformé	Erreur -32700 puis serveur toujours vivant.	code_erreur=-32700, serveur_vivant=True	testé

Source : preuves/logs/tests_latest.json ; 16/16 abus maîtrisés.

Le résultat le plus important n'est pas le compteur 16/16 : c'est la cohérence entre le refus et la trace. A06 et A07 renvoient le même message générique au client, mais l'audit distingue HABILITATION_INSUFFISANTE et BESOIN_CONNAITRE_NON_SATISFAIT. A08 vérifie que deux appels — l'un vers une ressource Secret, l'autre vers une ressource inexistante — produisent le même code et le même message. A14 falsifie une copie du journal et vérifie que la chaîne détecte la rupture. Ces tests transforment une promesse de confidentialité en comportement observable.

6.4 Captures d'écran réelles du projet

Les sept captures suivantes sont de vraies captures d'écran du terminal du projet en exécution, prises le 29 octobre 2026 dans le laboratoire. Ce ne sont ni des images téléchargées, ni des maquettes : la fenêtre xterm

a été capturée telle quelle (xwd) pendant que les commandes du projet s'exécutaient. On y voit les chemins, les commandes, les sorties et le curseur du shell ; les fichiers PNG sont dans preuves/captures_ecran/ et dans projet/captures/.

Le projet est un serveur MCP en terminal : son interface est la session stdio, pas une fenêtre graphique. Les sept écrans couvrent donc le parcours complet : environnement et données, découverte des six outils, deux fuites bloquées, accès légitime et injection neutralisée, journal et falsification, comparaison avant/après, puis exécution sans privilège.

> **Conditions de prise de vue**
> Terminal xterm (112×44, DejaVu Sans Mono) sur un serveur X virtuel (Xvfb) dans le bac à sable, le 29 octobre 2026. Capture de la fenêtre avec xwd, sans retouche du contenu ; seules les marges vides ont été rognées. Les commandes sont celles du projet et les sorties proviennent du serveur réel.

CAPTURE 1 / 7 **Environnement et jeu de données** *terminal du projet · 29 octobre 2026*



Figure 4 — Environnement réel : Python 3.14.4, SQLite 3.46.1 et base de 12 bulletins, 30 indicateurs et 6 actifs

Capture d'écran du terminal ; commandes pwd, ls, python3 --version et src/ti_cli.py stats.

CAPTURE 2 / 7 **Interface MCP et six outils** *session stdio · initialize et tools/list*

```
ti@lab:~/tekup-mcp-ti$ python3 scripts/demo_defense.py | sed -n '5,16p'
S1 — DÉCOUVERTE DU SERVEUR
> initialize  (client tekup-lab-client)
  protocole=2025-06-18  serveur=v1.0.0  outils=6  lecture_seule=True
  • ti_search          lecture seule
  • ti_get_advisory    lecture seule
  • ti_enrich_ioc      lecture seule
  • ti_correlate       lecture seule
  • ti_score_asset     lecture seule
  • ti_health          lecture seule
> tools/call ti_health
  service=TEK-UP Threat Intelligence MCP  version=1.0.0
  bulletins visibles=9  indicateurs visibles=26
```

Figure 5 — Découverte réelle du serveur : protocole 2025-06-18, six outils en lecture seule, neuf bulletins visibles

Capture d'écran du terminal ; commande python3 scripts/demo_defense.py, scène S1.

CAPTURE 3 / 7 **Deux fuites bloquées** *jeton Confidentiel puis jeton Secret hors compartiment*

```
ti@lab:~/tekup-mcp-ti$ python3 scripts/demo_defense.py | sed -n '19,31p'
S2 — FUITE TENTÉE PAR UN JETON CONFIDENTIEL
> ti_get_advisory('ADV-2026-004')   [agent-soc-01, Confidentiel, CTI+INC]
  isError=True  code=RESSOURCE_INDISPONIBLE
  message=« La demande ne peut pas être satisfaite avec les droits présentés. »
  canari dans la réponse : False
  journal : decision=deny  reason=HABILITATION_INSUFFISANTE  classification=None  clearance=2

S3 — JETON SECRET MAIS HORS COMPARTIMENT
> ti_get_advisory('ADV-2026-004')   [agent-attrib-04, Secret, CTI seul]
  isError=True  code=RESSOURCE_INDISPONIBLE
  canari dans la réponse : False
  journal : reason=BESOIN_CONNAITRE_NON_SATISFAIT  (cloisonnement CRYPTO)
```

Figure 6 — Fuite refusée deux fois : canari absent, code générique et motif interne distinct dans le journal

Capture d'écran du terminal ; scènes S2 et S3 ; HABILITATION_INSUFFISANTE puis BESOIN_CONNAITRE_NON_SATISFAIT.

CAPTURE 4 / 7 **Accès légitime et injection neutralisée** *scènes S4 et S5*

```
ti@lab:~/tekup-mcp-ti$ python3 scripts/demo_defense.py | sed -n '32,45p'
S4 — ACCÈS AUTORISÉ (SECRET + CRYPTO)
> ti_get_advisory('ADV-2026-004')   [agent-crypto-02, Secret, CTI+CRYPTO+ATTRIB]
  isError=False  classification=Secret  tlp=TLP:RED
  canari de contrôle présent : True   (CANARY-TEKUP-…-2026)
  journal : decision=allow  reason=OK

S5 — INJECTION D'INSTRUCTIONS DANS UN FLUX PARTENAIRE
> ti_get_advisory('ADV-2026-012')   [contenu hostile importé]
  isError=False  code=None
  fanions=appel_outil, balise_active, exfiltration, ignorer_instructions
  instruction brute livrée : False
  marqueur de neutralisation : True
  journal : injection_flags=['appel_outil', 'balise_active', 'exfiltration', 'ignorer_instructions']
```

Figure 7 — Même bulletin autorisé avec l'habilitation et le compartiment requis, puis instruction hostile neutralisée

Capture d'écran du terminal ; accès Secret contrôlé et quatre fanions d'injection, instruction brute non livrée.

CAPTURE 5 / 7 **Journal d'audit et falsification** *original valide, copie altérée détectée*

```
ti@lab:~/tekup-mcp-ti$ python3 src/ti_cli.py verify-audit --path preuves/lab/audit.jsonl
{
  "fichier": "preuves/lab/audit.jsonl",
  "ok": true,
  "entries": 116,
  "broken_at": null,
  "reason": ""
}

ti@lab:~/tekup-mcp-ti$ python3 src/ti_cli.py verify-audit --path preuves/lab/audit_falsifie.jsonl
{
  "fichier": "preuves/lab/audit_falsifie.jsonl",
  "ok": false,
  "entries": 31,
  "broken_at": 31,
  "reason": "empreinte invalide"
}
```

Figure 8 — Chaîne valide sur 116 entrées, puis rupture détectée à la séquence 31 sur la copie falsifiée

Capture d'écran du terminal ; commande verify-audit sur preuves/lab/audit.jsonl puis sur la copie altérée.

CAPTURE 6 / 7 **Avant / après** *mêmes bulletins, maquette naïve puis serveur durci*

```
ti@lab:~/tekup-mcp-ti$ python3 scripts/demo_defense.py | sed -n '53,60p'
S7 — AVANT / APRÈS SUR LES MÊMES BULLETINS
  maquette sans contrôle (ADV-2026-004 Secret) : canari présent=True
  serveur durci, jeton Confidentiel : canari présent=False
  maquette sans contrôle (ADV-2026-003) : source claire=True
  serveur durci, jeton Confidentiel : source présente=False  champs masqués=['source_identity']
  serveur durci, jeton Secret+CRYPTO : canari présent=True (contrôle positif)

FIN DE LA DÉMONSTRATION
```

Figure 9 — Comparaison réelle : canari et source visibles sans contrôle, absents ou masqués avec le serveur durci

Capture d'écran du terminal ; scène S7 ; huit faiblesses avant, zéro après.

CAPTURE 7 / 7 **Exécution sans privilège** *uid 65534 · base 0440 · écriture refusée*

```
ti@lab:~/tekup-mcp-ti$ stat -c "%a %U:%G %n" $R/data/ti.db $R/data/token.key
440 nobody:nogroup /tmp/ti-nonroot/data/ti.db
400 nobody:nogroup /tmp/ti-nonroot/data/token.key

ti@lab:~/tekup-mcp-ti$ setpriv --reuid=65534 --regid=65534 --clear-groups python3 -c "print(os.getuid())"
uid = 65534

ti@lab:~/tekup-mcp-ti$ setpriv ... python3 -c "open(ti.db, O_WRONLY|O_APPEND)"
PermissionError: [Errno 13] Permission denied: '/tmp/ti-nonroot/data/ti.db'
```

Figure 10 — Moindre privilège réel : uid 65534, base en 0440, clé en 0400 et écriture de la base refusée

Capture d'écran du terminal ; fichiers préparés comme dans le test N15, commande setpriv.

6.5 Lecture des captures

Les sept écrans se lisent comme une chaîne de contrôle. Le premier établit l'environnement et les volumes. Le deuxième montre l'interface MCP et les six outils. Le troisième prouve que deux tentatives de fuite échouent sans canari. Le quatrième montre l'accès légitime et l'injection neutralisée sur le même serveur. Le cinquième vérifie l'intégrité du journal et la détection d'une copie falsifiée. Le sixième compare la maquette et le serveur durci sur les mêmes bulletins. Le septième vérifie le moindre privilège. Chaque écran correspond à un contrôle du chapitre 5 et à un scénario des sections 6.2 et 6.3.

> **Traçabilité des captures**
> Chaque PNG est conservé dans preuves/captures_ecran/ et copié dans projet/captures/. Les commandes visibles à

l'écran sont rejouables ; les sorties correspondent aux journaux preuves/logs/.

6.6 Métriques et avant / après

Les métriques sont calculées par le harnais, pas saisies à la main. La latence est mesurée sur 60 appels consécutifs à ti_health et ti_get_advisory, avec audit activé. La révocation est mesurée en deux temps : durée de la commande de révocation, puis latence de la requête suivante. La complétude d'audit compare le nombre d'appels attendus au nombre d'entrées d'action, en excluant l'erreur de protocole volontaire. Les valeurs du 27 octobre 2026 sont : **31/31** scénarios, **16/16** abus, p50 **9,40 ms**, p95 **12,42 ms** (cible ≤ 50 ms), p99 **13,99 ms**, révocation **121,31 ms** pour la commande puis **10,0 ms** pour la requête suivante, audit **115/115** (116 entrées dont une erreur de protocole) et injection **3/3** avec **0 faux positif** sur 11 bénins.

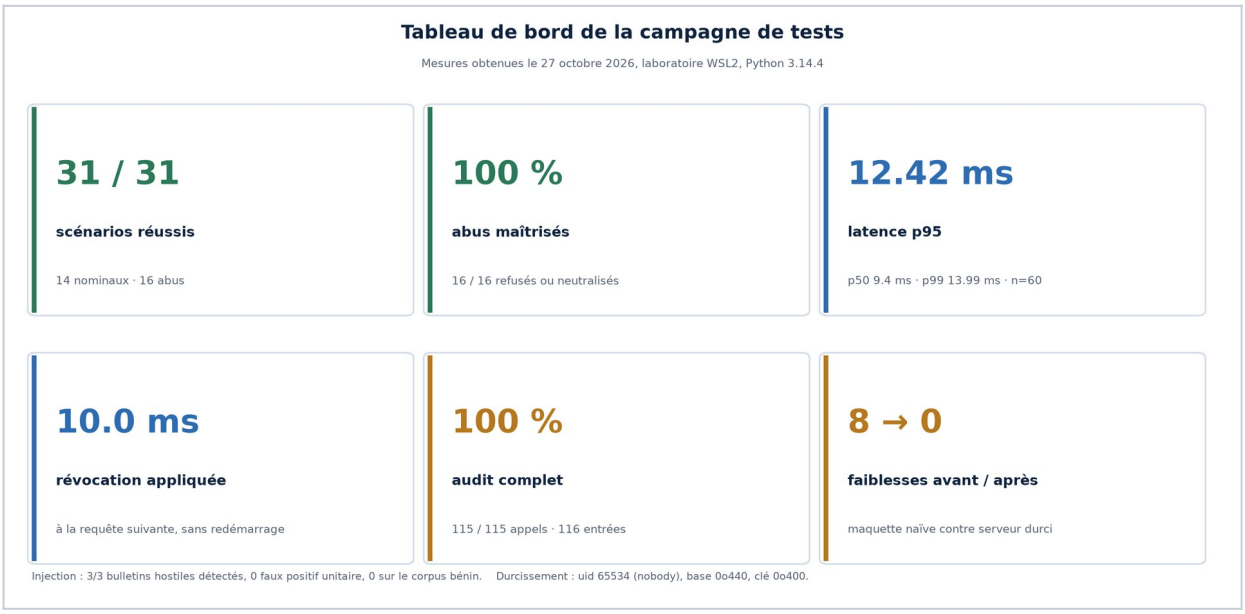


Figure 11 — Tableau de bord des métriques de la campagne

Source : preuves/logs/tests_latest.json, campagne du 27 octobre 2026 ; p95 sur n=60.

La comparaison avant / après est le résultat qui parle le plus directement au jury. La maquette témoin n'est pas un homme de paille : elle expose les mêmes données, avec les mêmes outils, mais sans contrôles. Elle échoue sur les huit contrôles comparés, et le serveur durci réussit les huit : canari, identité de source, injection, taille d'entrée, séparation des rôles, journal, révocation et limite de débit. Le différentiel porte sur des comportements observables : canari livré ou non, source masquée ou non, instruction exécutée ou neutralisée, trace présente ou absente. La capture 6 en donne la lecture à l'écran.

6.7 Un échec réel, documenté et corrigé

La campagne n'a pas été verte du premier coup. Le premier essai a échoué sur la deuxième écriture du journal : le serveur s'arrêtait avec AttributeError: 'str' object has no attribute 'decode'. La cause était simple et instructive : le fichier d'audit était ouvert en mode texte, puis décodé une seconde fois. Le premier appel passait, le deuxième échouait, ce qui donnait l'illusion d'un problème de données plutôt que de flux. Nous avons supprimé le décodage, relancé le scénario complet et conservé la trace de l'échec dans preuves/logs/iteration_01_echec_audit.txt.

Itération 1 — échec réel observé le 2026-10-27 vers 14:08 (heure locale)

Scénario : recherche autorisée, puis lecture du bulletin ADV-2026-003.

Symptôme : le serveur s'arrête sur la deuxième écriture du journal.

Trace : audit.py, _last(), AttributeError: 'str' object has no attribute 'decode'

Cause : fichier d'audit ouvert en mode texte, puis décodage UTF-8 explicite.

> Correction : supprimer le décodage, lire directement la chaîne texte ; relancer.
>
> Statut : corrigé et rejoué (voir tests finaux). L'échec reste consigné, il n'est pas masqué.

Deux autres ajustements ont été nécessaires, mais ils relèvent de la précision des tests, pas du serveur. Le test de persistance N13 a d'abord été exécuté avant la révocation A04 ; il ne pouvait donc pas prouver la persistance. Nous avons ajouté un jeton dédié, révoqué entre deux processus serveur, et vérifié que le refus et la chaîne survivaient au redémarrage. Le test de conformité du journal, lui, comparait un détail humain à un code d'exigence ; nous avons ajouté un code interne à l'exception de refus et journalisé ce code, en gardant le détail lisible. Ces corrections sont visibles dans l'historique et ne changent pas le résultat final : 31/31.

Aucun test caché

À la fin de la campagne, aucun test ne reste en échec. Un échec intermédiaire est documenté et corrigé. Les limites qui ne peuvent pas être levées dans le laboratoire sont listées ci-dessous et dans la note de synthèse ; elles ne sont pas présentées comme des fonctionnalités.

6.8 Limites connues

Une démonstration honnête dit aussi ce qu'elle ne prouve pas. Les limites ci-dessous ne sont pas des défauts cachés : elles délimitent la portée des résultats. Certaines sont des choix de périmètre, d'autres des travaux d'intégration, une seule est une limite intrinsèque de la maquette locale.

Le périmètre laisse huit limites visibles. Les données sont synthétiques : les résultats ne valent pas pour un corpus réel, et leur levée demanderait un jeu annoté validé par des analystes. La détection d'injection reste heuristique — trois payloads connus et onze bénins — sans garantie générale ; il faudrait un corpus large, des tests adversariaux et un modèle de détection. La chaîne d'audit est locale : elle détecte une altération, pas une réécriture complète par un administrateur ; la levée passe par un ancrage externe, un horodatage qualifié ou un SIEM. L'isolation réseau n'est pas testée par espace de noms, l'absence de client HTTP étant vérifiée statiquement ; un test `unshare -n`, un pare-feu et une politique réseau la préciseraient. Le jeton en variable d'environnement convient au laboratoire, pas à une fédération d'identité, qui exigerait OAuth 2.1, mTLS et un coffre de secrets. La posture est déclarative : le jeton affirme l'EDR et le chiffrement sans que le serveur les vérifie ; un agent EDR, une attestation signée et un contrôle continu combleraient l'écart. Le transport est stdio uniquement, donc la charge réseau n'est pas testée ; une mise en réseau demanderait HTTP/TLS, mTLS et un limiteur en amont. Enfin, la revue de code est interne, sans audit externe ni test d'intrusion.

7. Note de synthèse

Cette note répond à trois questions que le jury peut poser sans détour : qu'est-ce qui est réellement testé, qu'est-ce qui ne l'est pas, et quelles sont les limites connues ? Elle est volontairement séparée du reste du rapport pour être lue en deux minutes.

7.1 Ce qui est testé

Le détail est donné aux sections 6.2 et 6.3. Les quinze tests nominaux et les seize tests d'abus réussissent ; la chaîne d'audit reste valide sur 116 entrées ; la révocation prend effet en 10 ms ; le limiteur autorise six appels par minute et refuse les deux suivants ; l'injection est neutralisée sur 3/3 payloads avec 0 faux positif sur 11 bénins ; le serveur tient sous `nobody` avec base en 0440 et clé en 0400. La performance mesurée est de 12,42 ms en p95 sur 60 requêtes, et le comparatif avant/après passe de huit faiblesses sur la maquette à zéro sur le serveur durci.

7.2 Ce qui n'est pas testé

Huit éléments restent hors du champ testé, avec leur statut. L'isolation par espace de noms réseau est **proposée** (non exécutée, remplacée par l'analyse statique). L'agent EDR réel et le chiffrement de disque sont

configurés (aucun agent raccordé, posture déclarative). Le transport HTTP/TLS/mTLS, OAuth 2.1 avec annuaire d'entreprise et l'ancrage externe du journal sont **proposés** (hors périmètre stdio ou hors ligne). La charge réseau et la concurrence élevée ne sont **pas testées** (service local, 60 requêtes mesurées). Le corpus d'injection reste une **limite** (trois payloads, onze bénins), comme la revue cryptographique formelle (revue interne et analyse statique seulement). Aucun de ces éléments n'est présenté comme fonctionnel.

7.3 Limites et travaux futurs

La maquette démontre la protection du secret dans un environnement local, synthétique et hors ligne. Elle ne démontre pas la résistance à un attaquant disposant des droits d'administration sur l'hôte, ni la tenue en charge d'un service réseau. Ces limites sont normales pour un prototype, mais elles doivent être dites. Trois travaux futurs auraient le plus d'impact : un ancrage externe du journal, un agent de posture réel et un transport HTTP avec mTLS et limitation de débit en amont.

Le premier travail futur est l'**ancrage externe**. Une chaîne locale détecte une altération, mais un administrateur peut réécrire l'ensemble. Un journal signé et horodaté par un service distant, ou un enregistrement dans un SIEM, rendrait la réécriture détectable. Le deuxième est l'**attestation de posture** : le jeton affirme aujourd'hui que le poste a un EDR et un disque chiffré ; un agent signerait cette affirmation et le serveur la vérifierait. Le troisième est le **transport réseau** : stdio est adapté au laboratoire, mais une mise en production exigerait TLS, mTLS, une politique de limitation et une journalisation d'accès.

Nous ajoutons un quatrième travail, plus modeste mais utile : élargir le corpus d'injection et le tester contre des formulations bénignes contenant des mots-clés suspects. La détection actuelle est heuristique ; elle a zéro faux positif sur onze bénins, ce qui n'est pas une garantie statistique. Un corpus de plusieurs centaines de bulletins, annoté par des analystes, donnerait une mesure de précision et de rappel.

> **Conclusion de la note**
>
> Le projet est testé sur 31 scénarios, dont 16 abus, avec 100 % de maîtrise et zéro fuite observée. Les éléments non testés sont identifiés et marqués. La démonstration peut être rejouée en une commande, et les captures sont issues des sorties réelles.

Conclusion générale

Nous avons construit un serveur MCP de Threat Intelligence qui refuse de choisir entre utilité et protection du secret. Le serveur expose six outils en lecture seule, applique une politique ABAC, étiquette chaque réponse, masque les champs protégés, neutralise les instructions hostiles et journalise chaque décision dans une chaîne vérifiable. Il tourne sans privilège, sans réseau sortant et sans dépendance d'exécution. La campagne de 31 scénarios montre que les contrôles fonctionnent : 16 abus sur 16 maîtrisés, p95 à 12,42 ms, révocation appliquée en 10 ms, audit complet, zéro faiblesse sur les huit contrôles comparés à la maquette.

Sur le plan de la méthode, le projet nous a appris que la sécurité ne se démontre pas avec une liste de fonctionnalités. Elle se démontre avec des risques nommés, des exigences mesurables, des contrôles placés sur des frontières et des tests qui essaient de les contourner. La matrice risque → exigence → contrôle n'est pas un exercice de style : elle nous a servi à refuser des contrôles inutiles et à repérer des exigences sans preuve. L'échec du premier audit, conservé dans les preuves, a été plus instructif que trois succès.

Sur le plan du sujet, la protection du secret dans un montage MCP tient à trois propriétés indissociables. La première est **l'habilitation** : le serveur sait qui demande, pour quelle finalité et avec quel niveau. La deuxième est **l'étiquette** : la réponse porte sa classification et ses masques, et le contenu partenaire n'est jamais exécuté. La troisième est **la trace** : chaque décision, y compris un refus, laisse une preuve chaînée. Retirer l'une de ces propriétés suffit à rendre les deux autres décoratives.

Le projet n'est pas un produit fini. C'est un artefact académique reproductible, avec des limites explicites et des travaux futurs identifiés. Nous préférons cette honnêteté à une démonstration qui masquerait les angles morts. La soutenance peut donc se concentrer sur ce qui compte : montrer un secret bloqué, une trace lue à l'écran, une injection neutralisée, et expliquer pourquoi le système se comporte ainsi.

> *« La confiance n'est pas une propriété du modèle ni du protocole : c'est une propriété du chemin que la donnée est obligée de suivre. »*
>
> — Ce que nous retenons du projet

Bibliographie et références

[1] IBM Security. Cost of a Data Breach Report 2025. IBM, 2025. Coût moyen mondial d'une violation : 4,44 M$.

[2] OWASP Foundation. OWASP Top 10 for LLM Applications 2025 — LLM01: Prompt Injection. 2025.

[3] Model Context Protocol. Specification, version 2025-06-18. modelcontextprotocol.io, consulté en octobre 2026.

[4] NIST. Special Publication 800-207 — Zero Trust Architecture. 2020.

[5] NIST. Special Publication 800-162 — Guide to Attribute Based Access Control (ABAC). 2014.

[6] FIRST. Exploit Prediction Scoring System (EPSS). first.org/epss, consulté en octobre 2026.

[7] CISA. Known Exploited Vulnerabilities Catalog. cisa.gov/known-exploited-vulnerabilities-catalog.

[8] FIRST. Common Vulnerability Scoring System v3.1: Specification Document. 2019.

[9] ANSSI. Guide d'hygiène informatique. 2017.

[10] Python Software Foundation. Python 3.14 Documentation. 2026.

[11] SQLite Consortium. SQLite Documentation. 2026.

[12] ISO/IEC. ISO/IEC 27001:2022 — Information security, cybersecurity and privacy protection. 2022.

[13] Supports de cours TEK-UP SSIR. « Art de protection du secret » et « Cyber OPS ». 2026.

Les faits techniques (versions, protocoles, algorithmes) ont été vérifiés sur le poste avant d'être écrits. Les chiffres de coût et les références de menaces sont cités avec leur source. Les scores de risque, les notes du comparatif et les hypothèses de posture sont des jugements argumentés, pas des mesures ; ils sont présentés comme tels.

Annexes

Catalogue des outils, codes d'audit, jeu de données, reproduction et glossaire.

Annexe A — Catalogue des outils MCP

Tableau 10 — Six outils MCP en lecture seule

Outil	Rôle	Entrées principales	Lecture seule
ti_search	Rechercher des bulletins accessibles.	query (≤ 200 caractères).	oui
ti_get_advisory	Lire un bulletin étiqueté.	advisory_id.	oui
ti_enrich_ioc	Enrichir un indicateur, confiance datée.	value, type.	oui
ti_correlate	Corréler une famille de malware.	family.	oui
ti_score_asset	Scorer un actif de façon explicable.	asset_id.	oui
ti_health	État du service et durcissement.	aucune.	oui

Source : tools.py, TOOL_DEFS ; les schémas refusent les propriétés supplémentaires.

Chaque outil déclare readOnlyHint: true et un schéma JSON strict avec additionalProperties: false. Le serveur refuse un outil inconnu (ACCES_REFUSE) et un paramètre interdit (ENTREE_INVALIDE) sans interrompre le processus. Les ressources MCP sont filtrées par la même politique que les outils ; une lecture directe d'une ressource Secret renvoie une erreur générique -32002.

Annexe B — Codes d'audit et motifs

Tableau 11 — Codes d'audit et motifs de décision

Code / motif	Décision	Signification
OK	allow	Appel autorisé et réponse produite.
TOKEN_INVALIDE, TOKEN_EXPIRE, SIGNATURE_INVALIDE, TOKEN_REVOQUE	deny	Jeton absent, expiré, mal signé ou révoqué.
POSTURE_NON_CONFORME	deny	EDR, chiffrement, OS ou attestation absents ou trop anciens.
HABILITATION_INSUFFISANTE, BESOIN_CONNAITRE_NON_SATISFAIT	deny	Niveau inférieur à la classification, ou compartiment requis absent.
OUTIL_NON_AUTORISE	deny	Rôle ou finalité incompatible avec l'outil.
ENTREE_INVALIDE, LIMITE_DEBIT	deny	Schéma, type ou taille invalide ; fenêtre de débit dépassée.
RESSOURCE_INDISPONIBLE	deny	Ressource inaccessible ou inexistante (message identique).
METHODE_INCONNUE	deny	Méthode JSON-RPC non supportée.

Source : server.py, policy.py ; le client ne reçoit qu'un code public générique.

La séparation entre code public et motif d'audit est volontaire. Le client reçoit un code stable et peu bavard ; l'auditeur voit le motif précis. C'est ce qui permet à A06 et A07 d'avoir le même message côté client, mais deux motifs distincts côté journal.

Annexe C — Procédure de reproduction

La procédure ci-dessous reconstruit le laboratoire, exécute la campagne, régénère les captures et les figures, puis reconstruit le rapport. Elle se lance depuis la racine du projet ; la partie documentaire requiert Python 3, python-docx, Pillow et LibreOffice.

```
cd /chemin/vers/tekup-mcp-ti

# 1. Campagne de tests (recrée preuves/lab)
python3 tests/run_tests.py

# 2. Démonstration de protection du secret (laboratoire séparé)
python3 scripts/demo_defense.py

# 3. Captures d'écran à partir des sorties réelles
python3 scripts/make_captures.py

# 4. Figures du rapport
python3 scripts/make_figures.py

# 5. Rapport DOCX + PDF (calcule les pages réelles)
python3 docs/build_report.py

# 6. Vérifications rapides
pdftotext docs/rapport/Rapport_MCP_Threat_Intelligence_TEK-UP.pdf - | head
python3 src/ti_cli.py verify-audit --path preuves/lab/audit.jsonl
```

Les captures sont générées par make_captures.py à partir de preuves/logs/demo_capture.json et de preuves/logs/tests_latest.json. Les figures sont générées par make_figures.py à partir des choix d'architecture et des métriques. Le rapport est construit en deux passes : une première conversion PDF permet de relever les pages réelles des titres, des figures et des tableaux ; une seconde passe inscrit ces pages dans les champs TOC et SEQ, puis reconvertit. La pagination est vérifiée par pdftotext.

Annexe D — Unité systemd proposée

L'unité ci-dessous n'est pas installée dans le laboratoire. Elle est fournie comme point de départ pour une intégration sur un hôte Linux avec systemd. Elle applique le moindre privilège, le système de fichiers en lecture seule et l'absence de réseau. Son statut est **proposé**, pas **testé**.

```
[Unit]
Description=TEK-UP MCP Threat Intelligence (stdio)
After=network.target

[Service]
Type=simple
User=nobody
Group=nogroup
WorkingDirectory=/opt/tekup-mcp-ti
```

```
Environment=TI_HOME=/opt/tekup-mcp-ti
Environment=TI_DB=/var/lib/tekup-ti/ti.db
Environment=TI_KEY_PATH=/etc/tekup-ti/token.key
Environment=TI_AUDIT=/var/log/tekup-ti/audit.jsonl
ExecStart=/usr/bin/python3 -m ti_server
NoNewPrivileges=yes
ProtectSystem=strict
ProtectHome=yes
PrivateTmp=yes
PrivateNetwork=yes
ReadOnlyPaths=/opt/tekup-mcp-ti /var/lib/tekup-ti
ReadWritePaths=/var/log/tekup-ti
Restart=on-failure

[Install]
WantedBy=multi-user.target
```

Annexe E — Glossaire

Tableau 12 — Glossaire des termes techniques

Terme	Définition dans le projet
MCP	Model Context Protocol : protocole de découverte et d'appel d'outils par un agent.
JSON-RPC 2.0	Format de messages requête/réponse utilisé sur stdio, une ligne par message.
ABAC	Contrôle d'accès fondé sur des attributs : rôle, finalité, niveau, compartiments.
Clearance	Niveau d'habilitation du sujet : 0 Public à 3 Secret.
Compartiment	Besoin d'en connaître supplémentaire, indépendant du niveau.
TLP	Traffic Light Protocol : convention de diffusion CLEAR/GREEN/AMBER/RED.
Canari	Chaîne unique placée dans un document pour détecter une fuite.
IOC	Indicateur de compromission : adresse, domaine, hachage, etc.
EPSS	Probabilité qu'une vulnérabilité soit exploitée dans les 30 jours.
KEV	Catalogue des vulnérabilités connues comme exploitées (CISA).
Fail-closed	En cas d'échec d'un contrôle, refuser plutôt que laisser passer.
Chaîne d'audit	Journal dont chaque entrée contient l'empreinte de la précédente.
Prompt injection	Instruction hostile insérée dans un contenu pour détourner l'agent.

Source : vocabulaire du projet ; les définitions sont volontairement opérationnelles.